

Relatório de Auditoria de Segurança

Projeto atelie-erp — ERP/CRM de ateliê

04 de outubro de 2026 · Escopo: 5 categorias de auditoria

Stack verificada

TypeScript · Astro 7 SSR · Cloudflare Workers (@astrojs/cloudflare) · D1 (SQL via prepared statements, sem ORM) · Auth por sessão (cookie HttpOnly `crm_session` / Bearer, PBKDF2-SHA256 100k + salt por usuário) · Frontend sem framework (DOM imperativo) · Wrangler · WAHA (WhatsApp) em dev via docker-compose · CI GitHub Actions · sem Helm/Terraform/Docker de produção.

Metodológica — mapeamento das 5 categorias para a stack

1. Banco sem tranca (isolamento de dono) → RBAC + coluna de dono (OWNERSHIP) + SSE/LGPD.
2. Permissão no navegador → gates de UI vs. `requireRole/hasRole` no servidor.
3. IDOR → rotas de item/criação e ownership.
4. Chaves expostas → `.env/.dev.vars`, histórico git, CI, bundle, migrations, testes.
5. XSS → interpolações em `innerHTML`, `escapeText/escapeAtrib`, CSP, `set:html`.

Nota metodológica

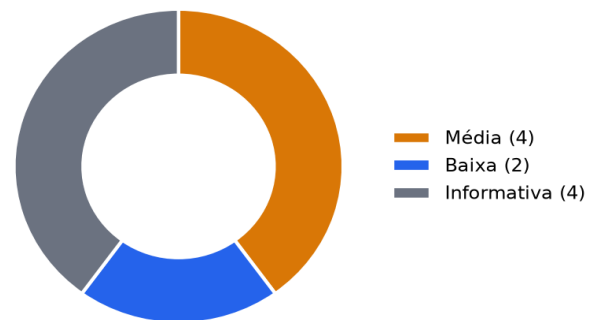
Somente achados verificados no código real (arquivo:linha + trecho + porquê + severidade). Nada de especulação. O que foi verificado e está CORRETO também é registrado (pontos fortes). Categorias parcialmente não aplicáveis são ditas explicitamente. Severidades: Conclusões da auditoria de 04/10/2026 — 10 achados (0 crítica, 0 alta, 4 média, 2 baixa, 4 informativa) + 18 pontos fortes.

1. Resumo executivo

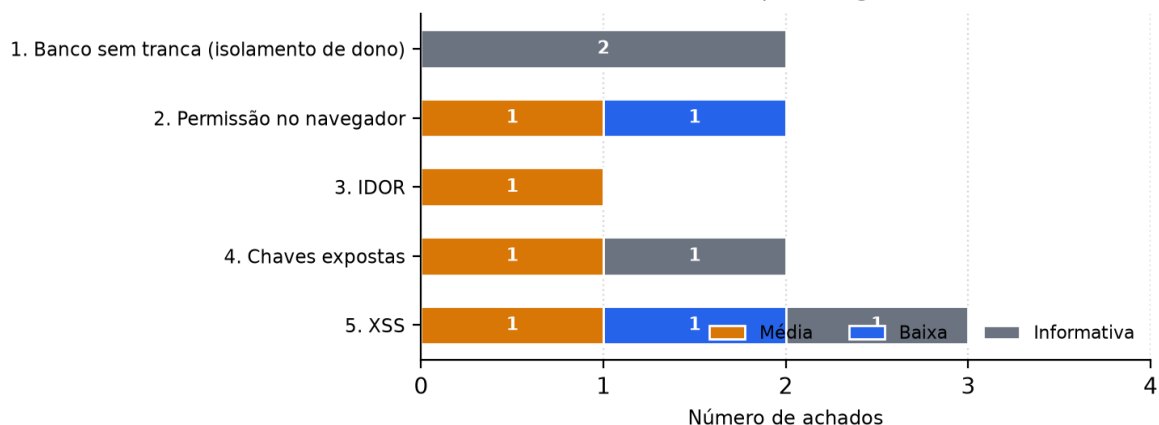
Auditoria completa das 5 categorias sobre o código real do repositório (backend, rotas de API, frontend, configs, histórico git e bundle). **Nenhum achado crítico ou alto.** 4 médios concentram-se em controles de autorização faltantes (webhook-config e criação de filhos), segredos no histórico git e id de cliente interpolado sem escape. As defesas estruturais — CSP, PBKDF2, rate limit, ownership fail-closed em escrita de item, gates de sessão — foram verificadas e estão corretas.

Severidade	Qtd	Categorias afetadas
Crítica	0	—
Alta	0	—
Média	4	2, 3, 4, 5
Baixa	2	2, 5
Informativa	4	1, 4, 5
Total	10	+ 18 pontos fortes verificados

Achados por severidade



Achados por categoria



Leitura: a categoria 2 (permissão) e a 5 (XSS) concentram a maioria dos achados acionáveis; a categoria 1 é majoritariamente informativa porque o app é single-tenant por decisão documentada; a 4 tem um achado de histórico git embora os segredos atuais estejam fora do repositório.

2. Pontos fortes verificados (o que está CORRETO)

Tudo abaixo foi lido e confirmado no código — não é lista de intenções:

✓ Escrita por item com posse	<code>routeFactory.ts:73-165</code>	✓ CSP + headers de segurança	<code>middleware.ts:31-46</code>
✓ Todas as 20 rotas [id] com guardItemWrite	<code>rotas */[id].ts</code>	✓ Admin semeado sem credencial default	<code>0004:5-9;</code> <code>bootstrap-admin.ts</code>
✓ Falha fechada para dono vazio	<code>routeFactory.ts:103-107</code>	✓ escapeText/escapeAtrib corretos (118 usos)	<code>domain/format.ts:33-47</code>
✓ SSE recorta conversas por dono p/ agent/viewer	<code>events.ts:25-40</code>	✓ Modal escapa título; bodyHtml interno escapado	<code>ui/Modal.ts:36</code>
✓ Envio WhatsApp valida posse da conversa	<code>send.ts:39-52</code>	✓ Sem eval/new Function/markdown lib/document.write	grep verificado
✓ Senha: PBKDF2-SHA256 100k + salt por usuário	<code>server/auth.ts</code>	✓ readWahaConfig rejeita placeholder (fail-closed)	<code>server/waha.ts:60-64</code>

✓ Anti-CSRF same-origin nas rotas de sessão	auth.ts (assertSameOrigin)	✓ Compose default sentinel + testes AC-120..123	docker-compose.waha. yaml:41
✓ Rate limit 5 login / 3 troca de senha	rateLimit.ts:13-15	✓ CI, dist e exemplos sem segredos reais	workflows; dist/ *.example
✓ Gate mustChangePassword em todas as rotas	middleware.ts:119-123	✓ Escopo LGPD por usuário (scopeForUser)	lgpdScope.ts:69-92

3. Achados por categoria (arquivo : linha)

Categoria 1 — Banco sem tranca (isolamento de dono/tenant)

Mecanismo identificado: single-tenant por design (README) + RBAC (authz.ts) + coluna de dono na escrita de item (OWNERSHIP, fail-closed). As listagens REST são globais por decisão documentada — por isso esta categoria só produz informativos, não vulnerabilidades.

ID	Severidade	Arquivo : linha	Descrição verificada
I1	Informativa	README.md src/server/routeFactory.ts:73-117 src/server/authz.ts	Single-tenant por design: listagens globais sem filtro por dono (decisão documentada); isolamento = RBAC + coluna de dono em escrita de item. Inconsistência: SSE (events.ts:25-40) recorta por dono, listagens REST não.
I2	Informativa	src/server/routeFactory.ts:236-255	createHandler só deriva o dono da sessão quando ausente — o POST aceita assignedUserId arbitrário do cliente (forja de atribuição).

Categoria 2 — Permissão no navegador

Comparação: UI/esconde vs. servidor/valida. Achados abaixo são casos em que o servidor NÃO valida o papel equivalente.

ID	Severidade	Arquivo : linha	Descrição verificada
F1	Média	src/pages/api/whatsapp/webhook-config.ts:25-27,57-59,72 src/middleware.ts:15-21	GET/PUT de webhook-config exigem apenas sessão (userFromToken), sem requireRole — qualquer papel (viewer) pode re-registrar/substituir os webhooks da sessão do engine (updateSession substitui a lista). A rota está em PUBLIC_PATHS, logo também pula o gate mustChangePassword. Rota irmã session.ts:51,70,116 exige ADMINS.
F4	Baixa	src/pages/api/users/[id].ts:30,88-93	PUT /api/users/:id: allowedRoles decide só pelo corpo da requisição (role/password → admin) e não pelo alvo — um manager pode alterar nome/e-mail de usuários admin (editablePatch).

Categoria 3 — IDOR

Rotas de item com guardItemWrite foram conferidas uma a uma (20 rotas) — o achado é na CRIAÇÃO (createHandler), única via sem checagem de pai.

ID	Severidade	Arquivo : linha	Descrição verificada
F2	Média	src/server/routeFactory.ts:236-264 src/pages/api/crm/messages/index.ts:3-6 conversation-notes/index.ts:6-9 activities/index.ts:6-9 src/pages/api/whatsapp/send.ts:39-52	POST de registros-filho sem validar a posse do pai: um agent pode gravar mensagem/nota/atividade em conversationId/contactId de outro agente. A checagem de pai (OWNERSHIP, kind='via') só roda em PUT/DELETE (guardItemWrite:152-165); send.ts:39-52 prova a intenção de posse por conversa (guardConversation).

Categoria 4 — Chaves expostas

Verificado: git log -S por chave/plaintext (nunca commitados), gitignored, exemplos com placeholder, CI/bundle limpos, sentinel + testes de compose. O achado é do HISTÓRICO removido.

ID	Severidade	Arquivo : linha	Descrição verificada
F6	Média	waha/.env (commits 882b050..59d917b) migrations/0004_crm_seed.sql (13 commits) tests/server/wahaComposeSecurity.test.ts:21,28	Segredos no histórico git: hash SHA-512 real da chave WAHA em waha/.env e como default de docker-compose.waha.yml (removido em 01/10/2026); hash PBKDF2 da senha padrão 'admin123' com sal fixo em 0004_crm_seed.sql; o teste anti-secredo ainda embute o hash antigo completo. Chave/plaintext ATUAIS nunca commitados (git log -S vazio).
I3	Informativa	.dev.vars:9 .dev.vars.example waha/.env.example src/server/waha.ts:60-64 .gitignore:5	Segredos fora do repositório: .dev.vars com plaintext real é gitignored e nunca commitado; exemplos só com placeholders; readWahaConfig rejeita dev_plaintext_change_me (fail-closed).

Categoria 5 — XSS

escapeText/escapeAtrib corretos (118 usos); sem eval/markdown lib; CSP script-src 'self'. Achados são interpolações que escaparam da convenção de escape.

ID	Severidade	Arquivo : linha	Descrição verificada
F3	Média	src/server/routeFactory.ts:36-40 ProductsView.ts:277 OrdersView.ts:272,312 CrmlnboxView.ts:451 CrmFunilView.ts:104,129 ComponentsView.ts:210 PurchasesView.ts:141 IngredientsView.ts:359	XSS armazenado via id: withNewId aceita body.id do cliente sem validar formato e os ids são interpolados em atributos HTML sem escapeAtrib (value=/data-id=). CSP script-src 'self' bloqueia a execução de JS inline, mas a injeção HTML/defacement persiste. Requer papel agent+ para criar.
F5	Baixa	src/ui/views/crm/CrmlnboxView.ts:410 src/ui/views/OrdersView.ts:67	value="`\${composerDelivery}`" e value="`\${day}`" sem escapeAtrib. Origem limitada a input[type=date]/todayISO (browser normaliza → não explorável via UI), mas viola a convenção do próprio arquivo (mesma função usa escapeText na linha 414).
I4	Informativa	src/pages/api/**/*. src/layouts/BaseLayout.astro:13,37	Parte servidor da categoria 5 não se aplica: respostas são JSON puro (sem templates HTML/e-mail com input do usuário); único set:html é constante estática (themeInitScript).

4. Recomendações

P1 (imediatas)

- Adicionar requireRole(ADMINS) (ou operador equivalente) em GET/PUT de /api/whatsapp/webhook-config e removê-la de PUBLIC_PATHS se não precisar ser pública.
- Validar a posse do pai em createHandler quando o OWNERSHIP declarar kind='via' (conversationId/contactId do autor ou papel elevado).
- Validar formato do id aceito em withNewId (ex.: `^[A-Za-z0-9_-]{1,64}$`) ou aplicar escapeAtrib em todos os value=/data-id= de id.

P2 (curto prazo)

- Rotacionar a chave WAHA antiga se ainda válida em algum ambiente; limpar o histórico git (git filter-repo) de waha/.env e do default de docker-compose.waha.yml; substituir o hash antigo literal do teste por um valor truncado.
- No PUT /api/users/:id, considerar o alvo do recurso em allowedRoles (manager não edita admin).
- Aplicar escapeAtrib nos value= de inputs de data (CrmlnboxView:410, OrdersView:67).

P3 (contínuo)

1. Unificar a intenção de isolamento: ou documentar que as listagens REST são globais por design, ou estender o recorte por dono do SSE às listagens.
2. Validar assignedUserId no POST (derivação obrigatória da sessão para papéis não elevados).
3. Teste de regressão: id malicioso rejeitado/escapado; POST filho em conversa alheia → 403; webhook-config sem papel → 403.

5. Issues para o GitHub (copiar/colar)

Agrupadas por causa-raiz para não gerar spam: 6 issues cobrem os 10 achados (F6 agrupa todo o histórico git).
Título na primeira linha, corpo entre os delimitadores abaixo; labels sugeridas: **security** + severidade (medium/low).

--- ISSUE 1 ---

TÍTULO: [Segurança] webhook-config sem verificação de papel (viewer pode re-registrar webhook)

CATEGORIA: 2 – Permissão no navegador / equivalente servidor

SEVERIDADE: Média

EVIDÊNCIA

- src/pages/api/whatsapp/webhook-config.ts:25-27 (GET) e :57-59 (PUT) chamam apenas userFromToken + 401; não há requireRole em nenhuma linha.
- src/pages/api/whatsapp/webhook-config.ts:72 – updateSession(config.session, [webhook]) SUBSTITUI a lista de webhooks da sessão no engine.
- src/middleware.ts:15-21 (linha 20) – rota listada em PUBLIC_PATHS, logo o middleware não anexa usuário e pula o gate mustChangePassword (middleware.ts:119-123).
- Contraste: src/pages/api/whatsapp/session.ts:51,70,116 exigem requireRole(ADMINS).

IMPACTO

Qualquer sessão autenticada (inclusive papel viewer, ou usuário bloqueado para troca de senha) pode ler o relatório de webhook e forçar a re-registracao, substituindo webhooks existentes da sessão do engine (perda de integrações).

CORREÇÃO SUGERIDA

- requireRole em GET/PUT (paridade com session.ts); remover de PUBLIC_PATHS se não for realmente pública; cobrir com teste de 403 para viewer.

CRITÉRIOS DE ACEITE

- GET/PUT sem papel adequado respondem 403 (viewer).
- Rota sai de PUBLIC_PATHS ou justificativa documentada.
- Teste automatizado cobrindo as permissões.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

TÍTULO: [Segurança] IDOR na criação de registros-filho (mensagens/notas/atividades em conversa alheia)

CATEGORIA: 3 – IDOR

SEVERIDADE: Média

EVIDÊNCIA

- src/server/routeFactory.ts:236-264 (createHandler): não valida posse do pai; em :249-255 só deriva owner de sessão para OWNERSHIP kind='column'.
- OWNERSHIP kind='via' (ex.: messages -> conversations) só é aplicado em guardItemWrite (:152-165), ou seja, apenas em PUT/DELETE.
- src/pages/api/crm/messages/index.ts:3-6, conversation-notes/index.ts:6-9, activities/index.ts:6-9: createCollectionRoutes com create:'agent'.
- Contraste: src/pages/api/whatsapp/send.ts:39-52 (guardConversation) exige assignedUserId === user.id ou papel elevado.

IMPACTO

Um agent autenticado pode gravar mensagem/nota/atividade em conversationId / contactId pertencente a outro agente (falsificação de histórico e poluição da conversa alheia), contornando a posse que o fluxo de envio já exige.

CORREÇÃO SUGERIDA

Em createHandler, quando o OWNERSHIP do recurso declarar kind='via', resolver o registro pai e exigir posse (autor ou manager/admin) antes do insert.

CRITÉRIOS DE ACEITE

- POST de message/note/activity em conversa de outro agent → 403 nao_autorizado.
- POST na própria conversa continua 201.
- Teste de regressão cobrindo os três endpoints.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

TÍTULO: [Segurança] XSS armazenado: id do cliente aceito sem validação e interpolado sem escape nos atributos

CATEGORIA: 5 — XSS

SEVERIDADE: Média

EVIDÊNCIA

- src/server/routeFactory.ts:36-40 (withNewId): aceita body.id arbitrário do cliente, sem validar formato, e persiste via insertEntity.
- Sinks sem escapeAtrib em atributos HTML:
 - src/ui/views/ProductsView.ts:277 value="\${item.id}"
 - src/ui/views/OrdersView.ts:272,312 value="\${c.id}" / value="\${p.id}"
 - src/ui/views/crm/CrmInboxView.ts:451 value=...
 - src/ui/views/crm/CrmFunilView.ts:104,129 data-deal-id=...
 - src/ui/views/ComponentsView.ts:210 value="\${i.id}"
 - src/ui/views/PurchasesView.ts:141 value="\${ing.id}"
 - src/ui/views/IngredientsView.ts:359 data-id="\${p.id}"
- Mitigação existente: CSP script-src 'self' (src/middleware.ts:31-34) bloqueia handlers inline e javascript:, impedindo execução de JS; sobra injeção HTML/defacement armazenado.

IMPACTO

Agent+ cria registro com id contendo aspas/HTML; ao abrir cadastros/pedidos/funil de outros usuários o HTML injetado é renderizado (defacement, phishing de UI, clobbering). Segurança depende apenas do CSP.

CORREÇÃO SUGERIDA

- Validar id no withNewId (ex.: `^[A-Za-z0-9_-]{1,64}$`) e rejeitar 400.
- Defesa em profundidade: escapeAtrib em todos os `value=/data-id=` de id.

CRITÉRIOS DE ACEITE

- POST com id fora do padrão → 400.
 - id legítimo continua funcionando (testes existentes verdes).
 - Varredura: nenhum `${...id}` cruo em atributo HTML.
- FIM ISSUE 3 ---

--- ISSUE 4 ---

TÍTULO: [Segurança] Chave WAHA e hash admin123 presentes no histórico git

CATEGORIA: 4 – Chaves expostas

SEVERIDADE: Média

EVIDÊNCIA

- waha/.env rastreado de 882b050 (2026-09-29) até 59d917b (2026-10-01, "T-103 feature: segredo da WAHA fora do controle de versao"):
WAHA_API_KEY_SHA512=sha512:b6eb06b35fa2...
- waha/docker-compose.waha.yml (histórico) usava o mesmo hash como default de \${WAHA_API_KEY_SHA512:-sha512:b6eb06b35fa2...}.
- migrations/0004_crm_seed.sql (13 commits) continha hash PBKDF2 de 'admin123' com sal fixo 'deskcomm-seed-v1'; removido no HEAD (comentários 0004:5-9).
- tests/server/wahaComposeSecurity.test.ts:21,28 ainda embutem o hash antigo completo como literal negativo.
- OK: chave ATUAL e plaintext nunca commitados (git log -S vazio); .dev.vars e waha/.env gitignored e untracked.

IMPACTO

O histórico permite offline brute-force do hash antigo (se a chave era fraca) e o hash 'admin123' confirma credencial default em bases que rodaram as migrations antigas; instâncias antigas podem ainda ter o usuário seed.

CORREÇÃO SUGERIDA

- Rotacionar a chave WAHA caso a antiga exista em qualquer ambiente.
- git filter-repo para remover waha/.env e o default antigo do compose.
- Trocar o literal do teste por hash truncado; verificar bases legadas por usuários seed (0019/0021 já tratam).

CRITÉRIOS DE ACEITE

- git log -S <trecho do hash antigo> vazio após a limpeza.
- Chave rotacionada documentada.
- Testes verdes sem o hash completo no repositório.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

TÍTULO: [Segurança] PUT /api/users/:id: manager pode editar nome/e-mail de usuários admin

CATEGORIA: 2/3 – Permissão / IDOR (classe de privilégio)

SEVERIDADE: Baixa

EVIDÊNCIA

- src/pages/api/users/[id].ts:30 – requireRole(context, allowedRoles(body)) decide só pelo corpo: role/password exigem admin; name/email caem em USER_MANAGERS.
- src/pages/api/users/[id].ts:88-93 (editablePatch) aplica body.name/body.email sem considerar o papel do usuário ALVO.

IMPACTO

Um manager altera nome/e-mail de contas admin (integridade de cadastro e vetores de phishing interno). role/password permanecem protegidos.

CORREÇÃO SUGERIDA

Recurso alvo admin só editável por admin (cheque de alvo em allowedRoles/patch).

CRITÉRIOS DE ACEITE

- manager → PUT em usuário admin (name/email) → 403.
- admin → PUT continua 200; manager editando agent continua 200.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

TÍTULO: [Segurança] Inputs de data interpolados em value= sem escapeAtrib

CATEGORIA: 5 – XSS (defesa em profundidade)

SEVERIDADE: Baixa

EVIDÊNCIA

- src/ui/views/crm/CrmInboxView.ts:410: value="\${composerDelivery}" (a mesma função usa escapeText no vizinho da linha 414).
- src/ui/views/OrdersView.ts:67: value="\${day}".
- Origem: todayISO()/input[type=date] (CrmInboxView:68,979; OrdersView:30,131) – o browser normaliza para yyyy-mm-dd, logo não explorável pela UI.

IMPACTO

Não explorável na prática hoje, mas quebra a convenção de escape do repositório e vira XSS se a origem mudar (ex.: estado setado por query param).

CORREÇÃO SUGERIDA

escapeAtrib nos dois value= (e varredura de interpolações em atributos).

CRITÉRIOS DE ACEITE

- Ambos os value= usam escapeAtrib; testes verdes.

--- FIM ISSUE 6 ---